

SIMATS ENGINEERING ASSESSMENT

TOOL 1

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

QUESTIONS:

Total Marks: 30

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly. **Questions:**

1. Identify appropriate **HTML5 semantic elements** missing in the structure.
2. Modify the structure to make it **standards-compliant**.
3. Interpret the **HTTP request generated** during form submission.
4. Analyze the **HTTP response cycle** from server to browser.
5. Suggest improvements for **usability and accessibility**.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
<head>
```

```
<title>Company</title>
</head>
<body>
  <div>
    <h1>Welcome
    <p>About us
  </div>
</body> </html>
```

Questions:

1. Identify **improper nesting and missing closing tags**.
2. Analyze how different browsers may interpret this structure.
3. Identify **missing semantic elements** (e.g., header, section).
4. Provide a **corrected W3C-compliant HTML structure**.
5. Suggest techniques to ensure **cross-browser compatibility**.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

Questions:

1. Identify issues in the **form configuration**.
2. Analyze why the **HTTP request is not triggered**.
3. Interpret the role of **GET method in this scenario**.
4. Propose a **corrected implementation**.
5. Suggest improvements for **secure data transmission**.

Code of Conduct:

*I **P Madhavi (192372133)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: P Madhavi

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

1)Question:

1. Identify the appropriate HTML5 semantic elements missing in the structure

The given HTML code contains only a <form> element. To make it a complete HTML5 document, the following semantic elements should be included:

- <!DOCTYPE html> – Specifies HTML5 document type.
- <html> – Root element of the webpage.
- <head> – Contains metadata and title.
- <title> – Displays the webpage title.
- <body> – Contains all visible content.
- <header> – Displays the page heading.
- <main> – Represents the main content.
- <section> – Groups related content.
- <label> – Provides labels for input fields.
- <footer> – Displays footer information.

These elements improve the structure, readability, accessibility, and search engine optimization (SEO) of the webpage.

2. Modify the structure to make it standards-compliant

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Online Symposium Registration</title>
</head>
<body>

<header>
  <h1>Online Symposium Registration Portal</h1>
</header>

<main>
  <section>
    <form action="/register" method="POST">

      <label for="name">Student Name:</label>
      <input type="text" id="name" name="name" required>

      <br><br>

      <label for="email">Email Address:</label>
      <input type="email" id="email" name="email" required>

      <br><br>
```

```
<button type="submit">Submit</button>

</form>
</section>
</main>

<footer>
  <p>&copy; 2026 College Symposium</p>
</footer>

</body>
</html>
```

3. Interpret the HTTP request generated during form submission

When the student clicks the **Submit** button, the browser sends an **HTTP POST request** to the server because the form uses `method="POST"`.

Example HTTP Request

POST /register HTTP/1.1

Host: www.college.edu

Content-Type: application/x-www-form-urlencoded

name=Madhavi&email=madhavi@gmail.com

Explanation

- **POST** – Sends form data securely to the server.
- **/register** – URL where the form data is submitted.
- **Host** – Specifies the server name.
- **Content-Type** – Indicates the format of submitted data.
- **Request Body** – Contains the student's name and email.

The browser waits for the server to process the request and send a response.

4. Analyze the HTTP response cycle from server to browser

The HTTP response cycle follows these steps:

1. The student enters the name and email.
2. The browser sends an HTTP POST request to the /register endpoint.
3. The web server receives the request.
4. The server validates the submitted information.
5. If the data is valid, it stores the registration details in the database.
6. The server prepares an HTTP response.
7. The browser receives the response and displays the registration result.

Example HTTP Response

HTTP/1.1 200 OK

Content-Type: text/html

```
<h2>Registration Successful!</h2>
```

Common HTTP Status Codes

- **200 OK** – Registration completed successfully.
- **400 Bad Request** – Invalid form data.
- **404 Not Found** – Requested page not found.
- **500 Internal Server Error** – Server encountered an error.

5. Suggest improvements for usability and accessibility

To improve the registration portal, the following enhancements can be made:

- Use `<label>` elements for all input fields.
- Add the required attribute for mandatory fields.
- Display meaningful validation error messages.
- Use semantic HTML5 elements such as `<header>`, `<main>`, `<section>`, and `<footer>`.
- Ensure keyboard accessibility for all controls.
- Use responsive design with the viewport meta tag.
- Maintain proper color contrast for better readability.
- Add placeholders or helper text to guide users.
- Validate data on both the client side and server side.
- Use descriptive button text such as **"Register Now"** instead of a generic **"Submit"**.

2)Question:

1. Identify improper nesting and missing closing tags

The given HTML code contains several syntax errors that make it invalid.

Errors Identified

- The `<h1>` tag is not closed (`</h1>` is missing).
- The `<p>` tag is not closed (`</p>` is missing).
- The HTML document does not include the `<!DOCTYPE html>` declaration.
- Semantic HTML5 elements such as `<header>`, `<main>`, and `<section>` are missing.
- The content is placed inside a generic `<div>` instead of meaningful semantic elements.

These issues make the HTML invalid and may lead to inconsistent rendering across browsers.

2. Analyze how different browsers may interpret this structure

Different web browsers use their own HTML parsing engines to handle invalid HTML code.

- Google Chrome (Blink Engine): Automatically closes missing tags and attempts to display the page correctly.

- Mozilla Firefox (Gecko Engine): Corrects the HTML structure but may produce a slightly different layout.
- Microsoft Edge (Blink Engine): Behaves similarly to Chrome by automatically fixing minor HTML errors.
- Safari (WebKit Engine): Repairs the HTML structure but may render spacing or formatting differently.

Because the HTML is not properly structured, browsers make their own assumptions, resulting in inconsistent layouts and rendering.

3. Identify missing semantic elements

The webpage should include the following HTML5 semantic elements:

- `<!DOCTYPE html>` – Declares HTML5.
- `<header>` – Contains the page heading.
- `<main>` – Contains the main content.
- `<section>` – Groups related content.
- `<footer>` – Contains footer information.

Using semantic elements improves accessibility, readability, maintainability, and search engine optimization (SEO).

4. Provide a corrected W3C-compliant HTML structure

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Company</title>
</head>

<body>

<header>
  <h1>Welcome</h1>
</header>

<main>
  <section>
    <p>About us</p>
  </section>
</main>
```

```
<footer>
  <p>&copy; 2026 Company</p>
</footer>
```

```
</body>
</html>
```

This version follows W3C HTML5 standards by including proper document structure, semantic elements, and correctly closed tags.

5. Suggest techniques to ensure cross-browser compatibility

To ensure that a website works consistently across different browsers, the following techniques should be followed:

- Write valid HTML5 and CSS according to W3C standards.
- Always close HTML tags properly and use correct nesting.
- Use semantic HTML5 elements such as <header>, <main>, and <section>.
- Include the <!DOCTYPE html> declaration.
- Test the website in multiple browsers such as Chrome, Firefox, Edge, and Safari.
- Use responsive design with the viewport meta tag.
- Avoid browser-specific features unless necessary.
- Use CSS resets or Normalize.css to maintain consistent styling.
- Validate HTML and CSS using W3C validation tools.
- Test the website on different screen sizes and devices.

Conclusion

Cross-browser compatibility can be achieved by writing valid HTML5 code, using semantic elements, properly closing tags, and testing the website across multiple browsers. Following W3C standards ensures consistent layout, improved accessibility, and a better user experience.

3) Question:

1. Identify issues in the form configuration

The given HTML form contains the following issues:

- The button is defined as type="button" instead of type="submit".
Therefore, clicking the button does not submit the form.

- The GET method is used to send login credentials, which is not secure because the data appears in the URL.
- The password field is sensitive information and should not be transmitted using the GET method.
- The form does not include client-side validation using the required attribute.

These issues prevent proper and secure form submission.

2. Analyze why the HTTP request is not triggered

The HTTP request is **not triggered** because the button type is:

```
<button type="button">Login</button>
```

A button with type="button" only performs actions when JavaScript is attached to it. It does **not** submit the form automatically.

To submit the form, the button should be:

```
<button type="submit">Login</button>
```

When the user clicks the **Submit** button, the browser generates an HTTP request and sends the form data to the server.

3. Interpret the role of GET method in this scenario

The **GET** method sends form data as part of the URL.

Example:

```
GET /submit?username=Madhavi&password=abc123 HTTP/1.1
```

Characteristics of GET

- Data is appended to the URL.
- Used for retrieving information from the server.
- Can be bookmarked and cached.
- Not suitable for sensitive information such as passwords because the data is visible in the browser's address bar.

For login forms, the **POST** method should be used instead of GET.

4. Propose a corrected implementation

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>Login Form</title>
```

```
</head>
```

```
<body>

<form action="/submit" method="POST">

  <label for="username">Username:</label>
  <input type="text" id="username" name="username" required>

  <br><br>

  <label for="password">Password:</label>
  <input type="password" id="password" name="password" required>

  <br><br>

  <button type="submit">Login</button>

</form>

</body>
</html>
```

This implementation correctly submits the form using the **POST** method and includes basic validation.

5. Suggest improvements for secure data transmission

The following measures should be implemented to improve security:

- Use the **POST** method instead of GET for login forms.
- Use **HTTPS** to encrypt data during transmission.
- Add the required attribute to mandatory fields.
- Validate input on both the client side and server side.
- Store passwords securely using hashing algorithms such as **bcrypt**.
- Implement authentication and session management.
- Protect against SQL Injection using prepared statements.
- Protect against Cross-Site Request Forgery (CSRF) using CSRF tokens.
- Display meaningful error messages without revealing sensitive information.

Conclusion

The form fails because the button uses `type="button"`, which does not submit the form. Additionally, using the GET method exposes sensitive information in the

URL. Changing the button to type="submit", using the POST method, and implementing HTTPS and proper validation ensures secure and reliable form submission.